accenture

Orchestration with Amazon EKS

Kubernetes in the Cloud

Chapter Overview

01 Kubernetes in Context

02 Kubernetes Core Concepts

03 Amazon EKS Fundamentals

04 Deploying to EKS

05 Scaling and Observability

06 Security and Production Patterns

Copyright © 2026 Accenture. All rights reserved.

Kubernetes in Context

Section 1



What is Kubernetes?

Platform-Level View

- Kubernetes (K8s) is an open-source container orchestration platform
- It is NOT tied to any cloud provider — fully platform-agnostic
- Managed by CNCF (Cloud Native Computing Foundation)

How It Works

- Kubernetes is a control-plane driven system using reconciliation loops
- You declare desired state — Kubernetes continuously works to match reality to intent

Core Capabilities

- Scheduling, scaling, self-healing, service discovery, and rolling updates

Copyright © 2026 Accenture. All rights reserved.

Kubernetes Deployment Options

Managed Cloud Services

- EKS (AWS), GKE (Google), AKS (Azure) — control plane managed by the provider
- Faster setup, less operational overhead, integrated with cloud ecosystem

Self-Managed

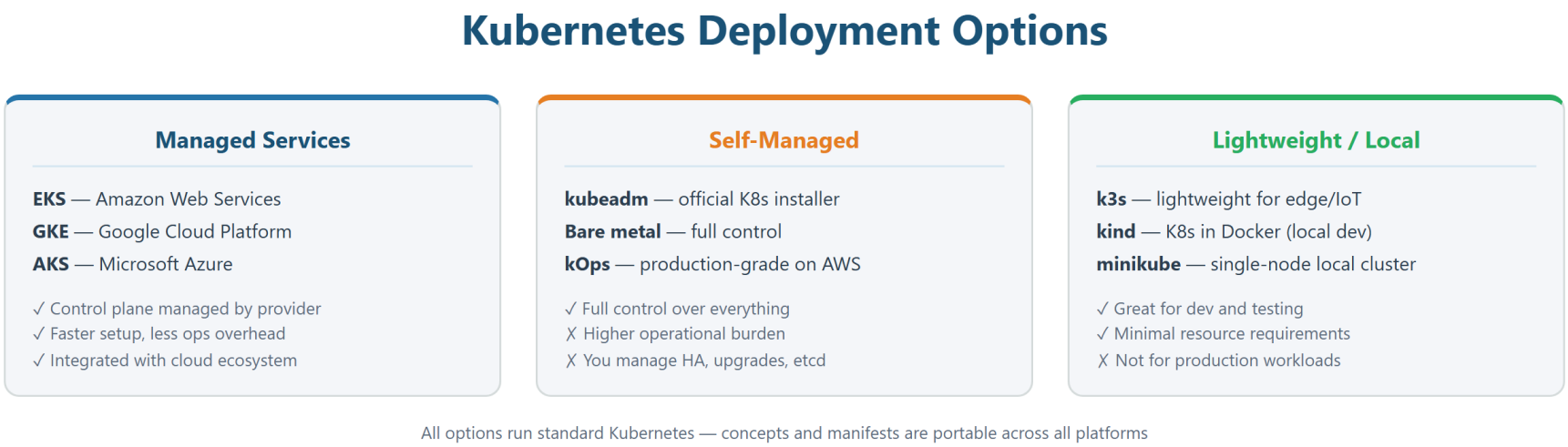
- kubeadm, bare metal, kOps — full control, higher operational burden
- You manage HA, upgrades, etcd backups, and infrastructure

Lightweight / Local

- k3s (edge/IoT), kind (K8s in Docker), minikube — great for dev and testing
- EKS is ONE option among many — Kubernetes concepts are portable

Copyright © 2026 Accenture. All rights reserved.

Kubernetes Deployment Options



ts reserved.

Why EKS?

AWS Ecosystem Integration

- IAM — fine-grained access control with IRSA (IAM Roles for Service Accounts)
- VPC networking — pods get real VPC IPs via the VPC CNI plugin
- ECR — private container registry with integrated image scanning
- CloudWatch, ALB/NLB, Secrets Manager — native observability and infrastructure

Important Distinction

- Kubernetes concepts (Pods, Deployments, Services) are portable across any K8s platform
- EKS-specific features (IRSA, VPC CNI, ALB Controller) are NOT portable

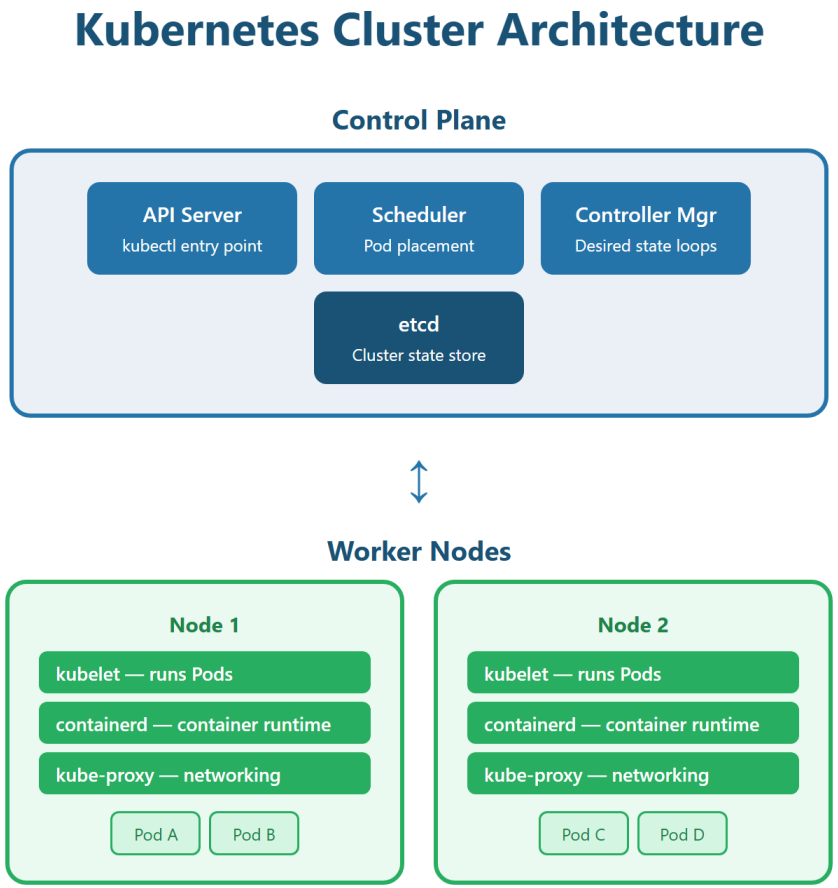
Copyright © 2026 Accenture. All rights reserved.

Kubernetes Core Concepts

Section 2



Kubernetes Cluster Architecture



Control plane manages cluster state; worker nodes run application containers via containerd (CRI-compatible runtime)

Copyright © 2026 Accenture. All rights reserved.

Control Plane Components

Control Plane

- The control plane is logically separate from the data plane (worker nodes)

API Server

- The ONLY entry point for all K8s operations — kubectl, dashboards, CI/CD, etc. all talk to this
- Validates and processes every request, stores results in etcd

Scheduler

- Watches for unscheduled pods, assigns them to nodes based on resource availability

Controller Manager

- Runs reconciliation loops — Deployment controller, ReplicaSet controller, Node controller

etcd

- Distributed key-value store holding the entire cluster state

Copyright © 2026 Accenture. All rights reserved.

Worker Node Components

kubelet

- Agent running on every node — receives pod specs from the API server, ensures containers are running

containerd (Container Runtime)

- CRI-compatible runtime that pulls images and manages container lifecycle
- containerd is the standard runtime for EKS and all major K8s distributions

kube-proxy

- Maintains network rules on each node so pods can communicate across the cluster

Copyright © 2026 Accenture. All rights reserved.

Core Resources

Pods

- Smallest deployable unit — one or more containers sharing network and storage
- Pods are ephemeral — should not store persistent data (use PersistentVolumes instead)

Deployments

- Declares desired state (replicas, image version) — controller handles rolling updates and rollbacks automatically

Services

- Stable network endpoint for a set of pods — ClusterIP (internal), NodePort, or LoadBalancer

ConfigMaps and Secrets

- Externalize configuration and sensitive data — injected as env vars or mounted files

Copyright © 2026 Accenture. All rights reserved.

Deployment Manifest

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: express-api
  namespace: production
spec:
  replicas: 3
  selector:
    matchLabels:
      app: express-api
  template:
    metadata:
      labels:
        app: express-api
    spec:
      containers:
        - name: api
          image: 123456789.dkr.ecr.us-east-1.amazonaws.com/express-api:v1.2.0
          ports:
            - containerPort: 3000
```



Copyright © 2026 Accenture. All rights reserved.

Essential kubectl Commands

```
# Apply manifests from a directory
kubectl apply -f k8s/

# List resources (namespace-aware)
kubectl get pods -n production
kubectl get deployments -n production
kubectl get services -n production

# Check rollout status
kubectl rollout status deployment/express-api -n production

# View logs (specific container in multi-container pod)
kubectl logs <pod-name> -c api -n production --follow

# Describe resource for events and status
kubectl describe pod <pod-name> -n production
```



Copyright © 2026 Accenture. All rights reserved.

Amazon EKS Fundamentals

Section 3



What is Amazon EKS?

Managed Kubernetes Service

- AWS runs the control plane — HA across 3 availability zones, automated etcd backups
- EKS manages ONLY the control plane — you are responsible for worker nodes and workloads

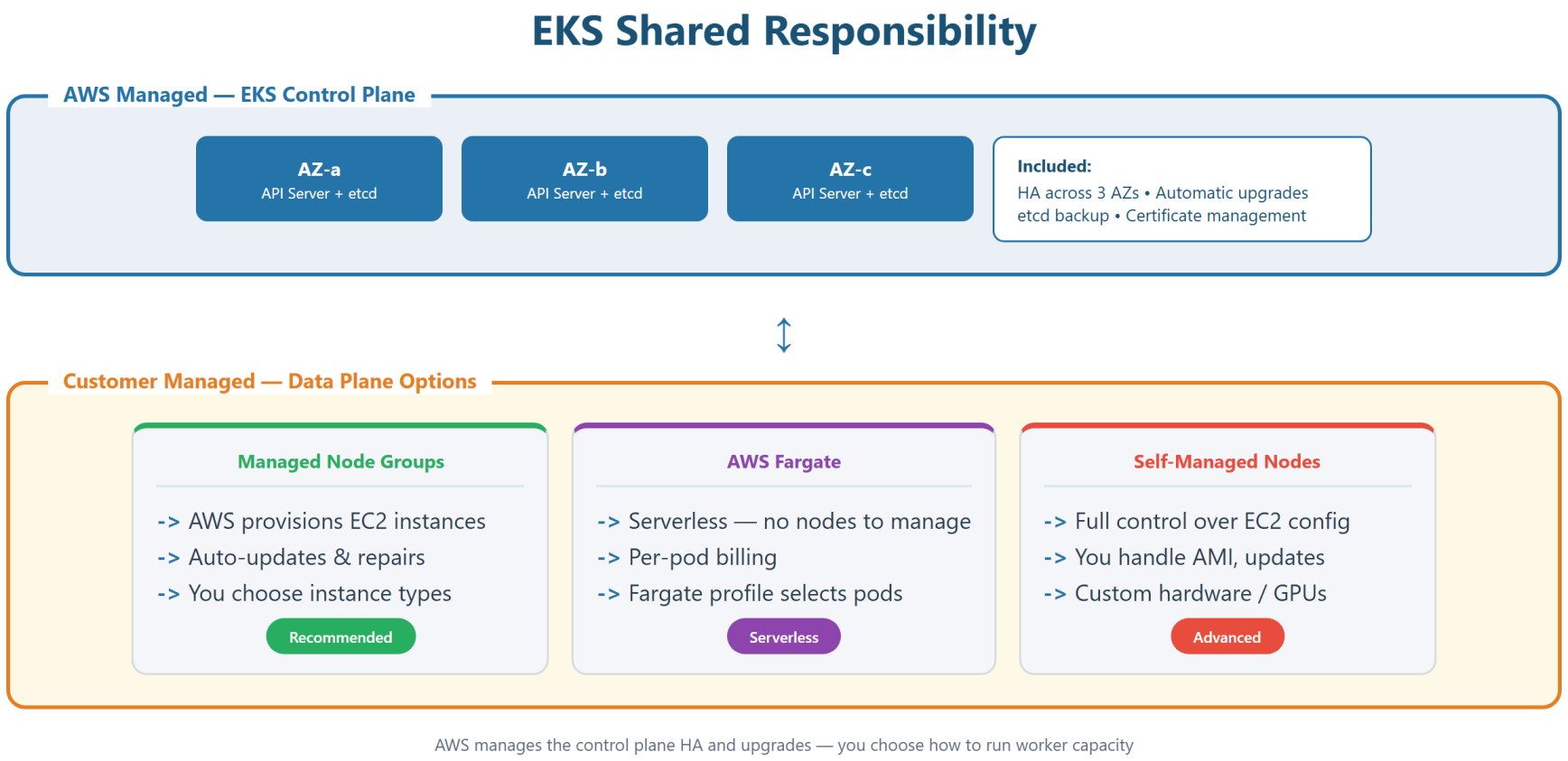
Key Tooling

- eksctl — CLI to create and manage EKS clusters and node groups
- aws eks update-kubeconfig — configures kubectl to authenticate with your cluster
- EKS add-ons — managed components like VPC CNI, CoreDNS, kube-proxy, and LB controller



Copyright © 2026 Accenture. All rights reserved.

EKS Shared Responsibility



AWS manages the control plane HA and upgrades — you choose how to run worker capacity

Copyright © 2026 Accenture. All rights reserved.

Data Plane Options

Managed Node Groups

- AWS provisions and auto-updates EC2 instances — you pick instance types and scaling
- Default recommendation for most workloads — balances control and convenience

AWS Fargate

- Serverless pods — no nodes to provision, patch, or scale
- Limitations: no DaemonSets, limited control over networking and storage
- Fargate profiles select which pods run serverless based on namespace and labels

Self-Managed Nodes

- Full control over EC2 AMI, launch templates, and configuration — for GPU or custom needs



Copyright © 2026 Accenture. All rights reserved.

EKS Networking Fundamentals

VPC CNI Plugin

- Each pod gets a real VPC IP address — no overlay, native AWS networking speed
- Pods communicate with other AWS services using standard VPC routing
- IP exhaustion can become a scaling bottleneck — requires careful subnet sizing

Subnet Design

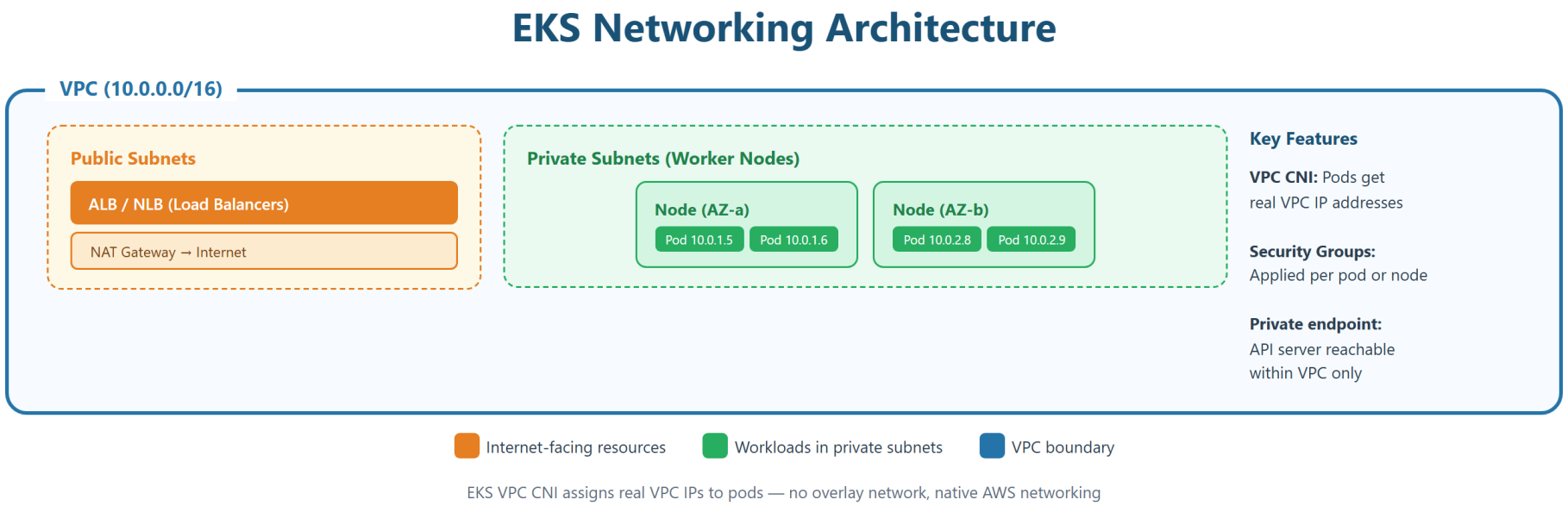
- Public subnets — load balancers and NAT gateways for outbound internet
- Private subnets — worker nodes and pods (best practice for production)

Security Groups

- Applied at the node level or per-pod with security groups for pods feature

Copyright © 2026 Accenture. All rights reserved.

EKS Networking Architecture



Copyright © 2026 Accenture. All rights reserved.

Amazon ECR — Container Registry

Private Docker Registry in AWS

- ECR stores Docker images encrypted at rest, integrated with IAM for access control
- Image scanning detects OS and package vulnerabilities on push

Authentication

- Docker clients must authenticate explicitly: aws ecr get-login-password | docker login
- EKS nodes pull images using the node IAM role — requires ecr:GetDownloadUrlForLayer permission

Best Practices

- Enable tag immutability to prevent overwriting — use semantic versioning or git SHA tags

Copyright © 2026 Accenture. All rights reserved.

ECR Push Workflow

```
# Authenticate Docker to ECR
aws ecr get-login-password --region us-east-1 \
  | docker login --username AWS \
    --password-stdin 123456789.dkr.ecr.us-east-1.amazonaws.com

# Create repository (first time only)
aws ecr create-repository --repository-name express-api \
  --image-scanning-configuration scanOnPush=true \
  --image-tag-mutability IMMUTABLE

# Build, tag, and push
docker build -t express-api:v1.2.0 .
docker tag express-api:v1.2.0 \
  123456789.dkr.ecr.us-east-1.amazonaws.com/express-api:v1.2.0
docker push \
  123456789.dkr.ecr.us-east-1.amazonaws.com/express-api:v1.2.0
```



Copyright © 2026 Accenture. All rights reserved.

Deploying to EKS

Section 4



Creating an EKS Cluster

```
# Create cluster with managed node group
eksctl create cluster \
  --name my-cluster \
  --region us-east-1 \
  --version 1.29 \
  --nodegroup-name workers \
  --node-type t3.medium \
  --nodes 2 \
  --nodes-min 2 \
  --nodes-max 5 \
  --managed \
  --with-oidc

# Configure kubectl
aws eks update-kubeconfig --name my-cluster --region us-east-1

# Verify connectivity
kubectl get nodes
```



Copyright © 2026 Accenture. All rights reserved.

End-to-End Deployment Workflow

1. Build and Push Image

- Build Docker image with multi-stage Dockerfile, tag with version, push to ECR

2. Write Kubernetes Manifests

- Deployment, Service, ConfigMap, and optionally Ingress YAML files in a k8s/ directory
- Resource requests and limits must always be defined in production manifests

3. Apply to Cluster

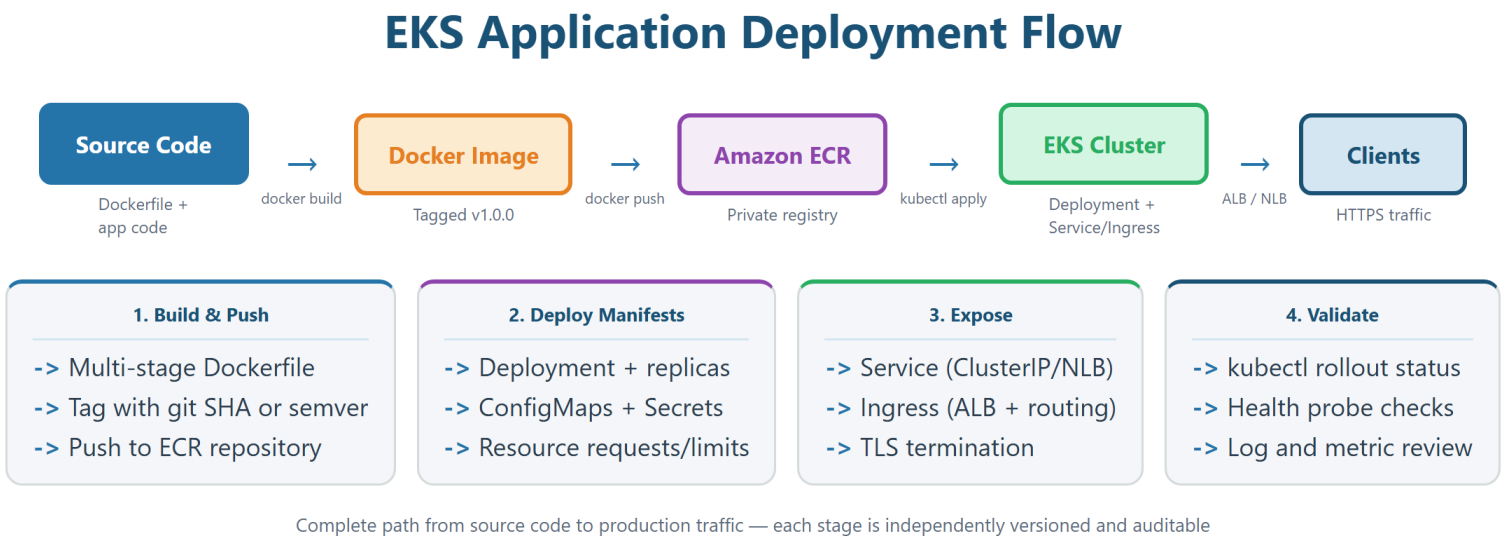
- kubectl apply -f k8s/ — applies all manifests, triggers rolling deployment

4. Validate Rollout

- kubectl rollout status, check pod readiness, review logs and events
- CI/CD pipelines should automate the entire build, scan, and deploy workflow

Copyright © 2026 Accenture. All rights reserved.

Application Deployment Flow



Copyright © 2026 Accenture. All rights reserved.

Production Deployment Manifest

```
spec:
  containers:
    - name: api
      image: 123456789.dkr.ecr.us-east-1.amazonaws.com/express-api:v1.2.0
      ports:
        - containerPort: 3000
      resources:
        requests:
          cpu: "250m"
          memory: "256Mi"
        limits:
          cpu: "500m"
          memory: "512Mi"
      readinessProbe:
        httpGet:
          path: /ready
          port: 3000
        initialDelaySeconds: 5
        periodSeconds: 10
      livenessProbe:
        httpGet:
          path: /healthz
          port: 3000
        initialDelaySeconds: 15
        periodSeconds: 20
      env:
        - name: NODE_ENV
          value: "production"
        - name: DB_PASSWORD
          valueFrom:
            secretKeyRef:
              name: app-secrets
              key: db-password
```



Copyright © 2026 Accenture. All rights reserved.

Exposing Applications

Service Types

- ClusterIP — internal-only (default), pods reach each other by Service DNS name
- LoadBalancer — provisions an AWS NLB by default for TCP/UDP traffic
- Each LoadBalancer Service creates a separate NLB — costs add up with many services

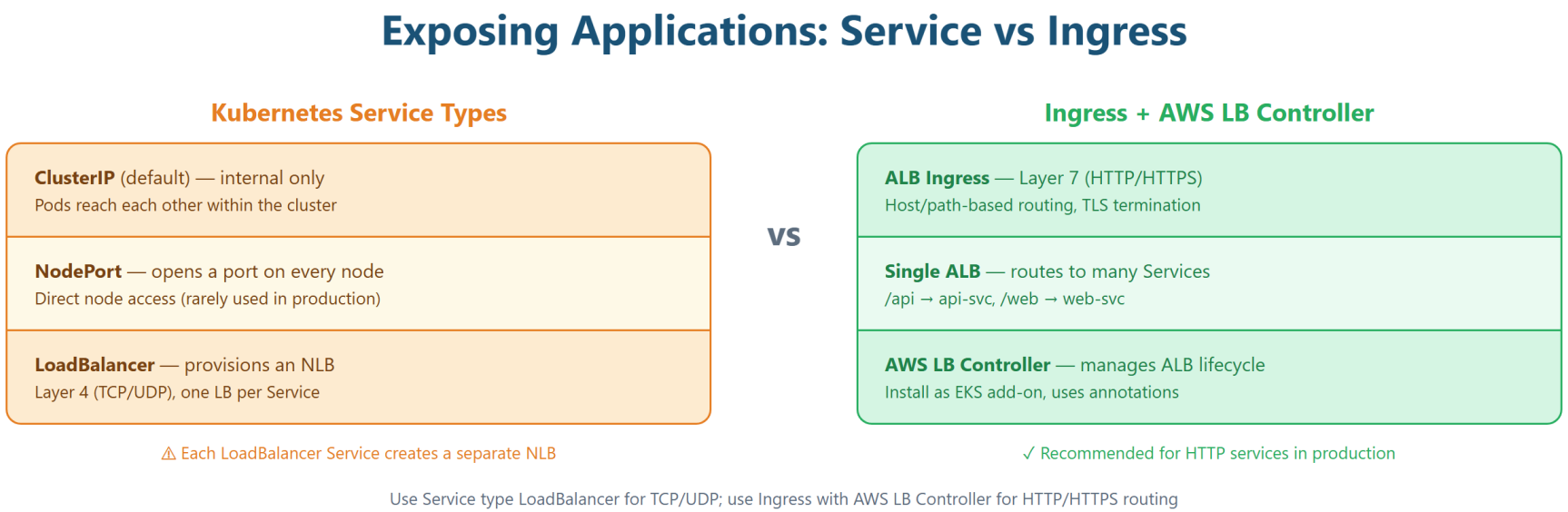
Ingress with AWS Load Balancer Controller

- Creates an Application Load Balancer (ALB) for HTTP/HTTPS with path-based routing
- One ALB can route to many backend services — /api, /web, /admin
- ALB Ingress Controller is AWS-specific — not portable to other K8s platforms



Copyright © 2026 Accenture. All rights reserved.

Service vs Ingress Exposure



Copyright © 2026 Accenture. All rights reserved.

Ingress with AWS Load Balancer Controller

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app-ingress
  annotations:
    alb.ingress.kubernetes.io/scheme: internet-facing
    alb.ingress.kubernetes.io/target-type: ip
    alb.ingress.kubernetes.io/certificate-arn: arn:aws:acm:...
spec:
  ingressClassName: alb
  rules:
    - host: api.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: express-api-service
                port:
                  number: 80
```



Copyright © 2026 Accenture. All rights reserved.

Scaling and Observability

Section 5



Horizontal Pod Autoscaler (HPA)

How HPA Works

- Monitors pod CPU/memory via metrics-server (must be installed as an EKS add-on)
- Compares current utilization against a target — scales replicas within a min/max range
- CPU-based scaling alone is often insufficient — consider custom metrics or KEDA for event-driven scaling

Resource Requests and Limits

- requests — minimum resources guaranteed (used for scheduling decisions)
- limits — maximum resources the pod can consume (enforced by cgroups)
- HPA targets are based on requests, not limits — accurate requests are critical

Copyright © 2026 Accenture. All rights reserved.

HPA and Resource Configuration

```
# Install metrics-server (required for HPA)
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/\
  releases/latest/download/components.yaml

# Create HPA for a deployment
kubectl autoscale deployment express-api \
  --namespace production \
  --min=2 \
  --max=10 \
  --cpu-percent=70

# Check HPA status
kubectl get hpa -n production
# NAME          REFERENCE              TARGETS   MINPODS   MAXPODS   REPLICAS
# express-api   Deployment/express-api  45%/70%   2         10        3
```



Copyright © 2026 Accenture. All rights reserved.

Health Probes in Detail

Liveness Probe

- Checks if the container is alive — failure triggers a container restart
- Catches deadlocks, infinite loops, and corrupted state

Readiness Probe

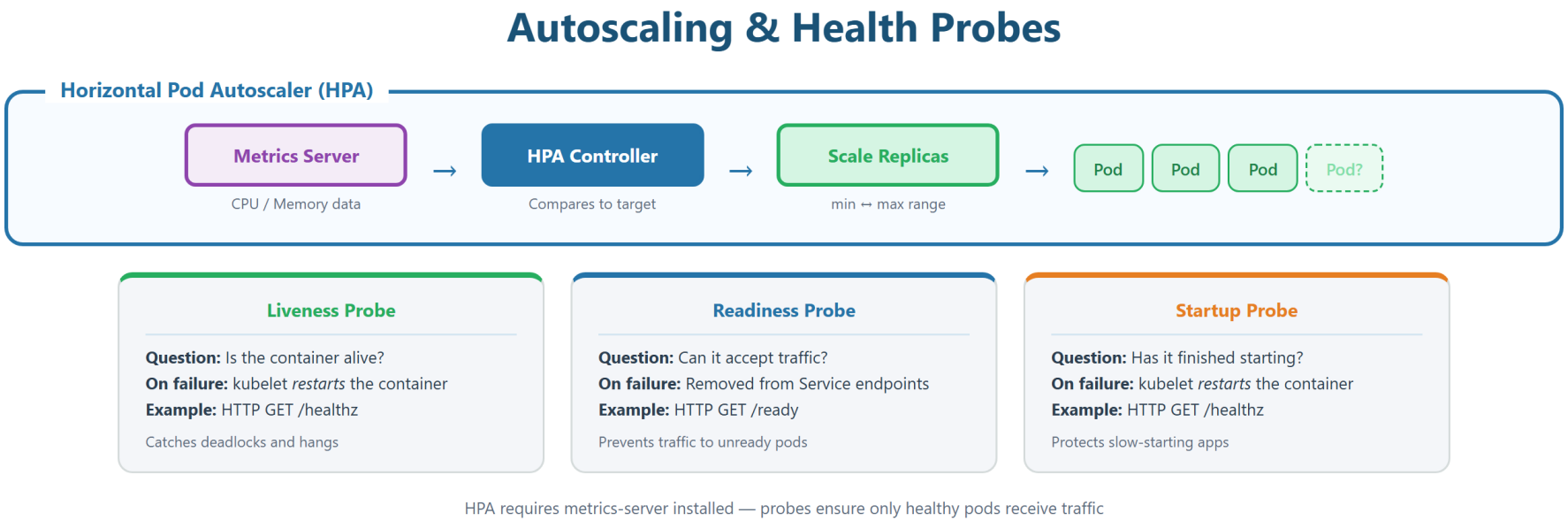
- Checks if the pod can accept traffic — failure removes it from Service endpoints
- Prevents routing to pods still warming up or temporarily overloaded

Startup Probe

- Protects slow-starting containers — liveness/readiness probes are disabled until startup succeeds

Copyright © 2026 Accenture. All rights reserved.

Autoscaling and Health Model



Copyright © 2026 Accenture. All rights reserved.

Cluster Observability

CloudWatch Container Insights (AWS-Specific)

- Collects cluster, node, pod, and container metrics — CPU, memory, network, disk
- Enabled as an EKS add-on with the CloudWatch agent DaemonSet

Log Collection

- Fluent Bit or Fluentd DaemonSet forwards container logs to CloudWatch Logs
- Control plane logs (API server, audit) can be sent to CloudWatch directly from EKS

Prometheus and Grafana (Portable)

- Amazon Managed Prometheus (AMP) for metrics, Amazon Managed Grafana for dashboards
- Prometheus/Grafana are platform-agnostic — work on any Kubernetes cluster

Copyright © 2026 Accenture. All rights reserved.

Security and Production Patterns

Section 6



IAM Roles for Service Accounts (IRSA)

The Problem

- Pods need AWS permissions (S3, DynamoDB, SQS) but shouldn't use node-level IAM roles
- Node roles grant every pod on that node the same permissions — violates least privilege

The Solution: IRSA

- Associate a specific IAM role with a Kubernetes ServiceAccount
- Only pods using that ServiceAccount get those AWS permissions
- Uses OIDC federation — cluster must be created with --with-oidc
- IRSA is AWS-specific — other platforms have their own identity models (GKE Workload Identity, AKS Pod Identity)

Copyright © 2026 Accenture. All rights reserved.

RBAC and Access Control

Kubernetes RBAC (Portable)

- Role — defines permissions (get, list, create, delete) on resources within a namespace
- RoleBinding — grants a Role to a user, group, or ServiceAccount
- ClusterRole / ClusterRoleBinding — same concept, cluster-wide scope
- RBAC is Kubernetes-native and portable — works the same on any K8s platform

EKS Access Management (AWS-Specific)

- EKS access entries map IAM principals to Kubernetes RBAC groups
- Replaces the older aws-auth ConfigMap approach for cluster authentication



Copyright © 2026 Accenture. All rights reserved.

Kubernetes Secrets

How Secrets Work

- Kubernetes Secrets store sensitive data separately from pod specs and images
- Injected as environment variables or mounted as files in the container

Important Limitations

- Secrets are base64 encoded, NOT encrypted by default — anyone with API access can read them
- Enable etcd encryption at rest for basic protection

External Secret Stores

- Use AWS Secrets Manager or HashiCorp Vault for production secrets management
- External Secrets Operator syncs external secrets into Kubernetes Secret objects

Copyright © 2026 Accenture. All rights reserved.

Secrets and RBAC Configuration

```
# Create a Secret from literal values
kubectl create secret generic app-secrets \
  --namespace production \
  --from-literal=db-password=S3cur3P@ss \
  --from-literal=api-key=abc123xyz

# RBAC: Role for read-only access to pods
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: production
  name: pod-reader
rules:
  - apiGroups: [""]
    resources: ["pods", "pods/log"]
    verbs: ["get", "list", "watch"]
```



Copyright © 2026 Accenture. All rights reserved.

Troubleshooting EKS

First Steps: Always Check Events

- kubectl describe pod <pod> — events section shows scheduling, pulling, and startup issues
- Logs are per container — specify -c <container> in multi-container pods

Pod-Level Issues

- ImagePullBackOff — wrong image name, missing ECR permissions, or tag not found
- CrashLoopBackOff — app crashes on start, check logs: kubectl logs <pod>
- Pending — insufficient node resources, check events for scheduling failures

Network and EKS-Specific Issues

- Service not reachable — verify selector labels match pod labels
- Auth errors — verify IAM role, OIDC provider, and EKS access entries

Copyright © 2026 Accenture. All rights reserved.

EKS Troubleshooting Commands

```
# Check pod status and events
kubectl get pods -n production -o wide
kubectl describe pod <pod-name> -n production

# View logs (follow and filter)
kubectl logs <pod-name> -n production --follow --tail=100

# Shell into a running container
kubectl exec -it <pod-name> -n production -- sh

# Check cluster events (sorted by time)
kubectl get events -n production --sort-by=.metadata.creationTimestamp

# Verify node health and capacity
kubectl get nodes -o wide
kubectl top nodes
kubectl top pods -n production
```



Copyright © 2026 Accenture. All rights reserved.

Key Takeaways

Key Takeaways

1. Kubernetes is platform-agnostic orchestration — concepts and manifests are portable
2. EKS is one managed Kubernetes offering — it manages the control plane, you manage workloads
3. Kubernetes concepts (Pods, Deployments, Services, RBAC) work on any K8s platform
4. EKS integrates deeply with AWS — IRSA, VPC CNI, ALB Controller, CloudWatch
5. Resource management and observability are critical — set requests/limits, monitor with Prometheus
6. Production requires security and automation — RBAC, external secrets, CI/CD pipelines



Copyright © 2026 Accenture. All rights reserved.